Clase 17 · Algoritmos de planificación de tareas y CFS de Linux

Fundamentos de Sistemas Embebidos · M.C. José Mauricio Matamoros de María y Campos · lunes 26 oct 2026 · Tema 3 (3.3) · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Fuente

NotebookLM (audio FSE-Clase17). Retoma lo pospuesto en la Clase 16. ⚠️ marca datos que NotebookLM dictó y que no cuadran con la referencia técnica: confirmar antes del examen.

1. El scheduler

Componente del kernel que administra el **procesador**: decide qué proceso, hilo o tarea en **READY** obtiene el núcleo. El SO administra procesador (scheduler), memoria (asignación) y periféricos (drivers). Tarea, hilo y proceso son **intercambiables** para el scheduler. Una tarea en **WAIT/BLOCKED** no puede entrar al núcleo.

Error común: darle CPU a una tarea que espera datos del puerto serie o del teclado.

2. Algoritmo 1: FIFO simple (no preventivo)

Orden estricto de llegada; cada tarea corre a completitud.

1. Tareas 1, 2 y 3 en READY; la 1 pasa a RUNNING.
2. La 1 necesita datos de un periférico o de la 2 → WAIT.
3. Corre la 2, genera los datos y termina.
4. La 1 se desbloquea a READY y se encola **al final**: no regresa hasta que corra la 3.

- En embebidos es el **80–90 %** de la lógica: un bucle `while(1)`. Es el de menor *overhead* (firmware bare-metal).
- Riesgo: un bucle infinito en la primera tarea bloquea todo.

3. Algoritmo 2: Shortest Job First (no preventivo)

Elige la tarea en READY con el **menor tiempo estimado**. Se organiza en un **árbol** (NotebookLM: "Árbol B") ordenado de menor a mayor; la más corta queda arriba.

Ejercicio del pizarrón (300 tareas)

Búsqueda lineal: hasta 300 comparaciones, O(N). Árbol: $O(\log_2 N)$; $\log_2(300) \approx 8.22$ → **9 comparaciones**. Reordenar al insertar: O(N log N).

- No se puede predecir con exactitud el tiempo de una tarea (depende de datos y condicionales); se usa el **peor caso, WCET** (*Worst-Case Execution Time*).
- Evita que tareas breves (vaciar búferes UART/I²C) queden detrás de cálculos pesados.

4. Algoritmo 3: prioridades e inanición

Despacha siempre la tarea lista de **mayor prioridad**.

- Escala de Linux (*niceness*): −20 a 20 (⚠️ en Linux real, −20 a 19); **−20 es la más alta**, 20 la más baja; se ordena de menor a mayor.
- **Preventivo:** al activarse una tarea de mayor prioridad, el kernel interrumpe a la de menor.
- **Inanición (starvation):** con la Tarea 1 alta, 2 y 3 medias y 4 baja, si las tres primeras se bloquean y desbloquean sin parar, la Tarea 4 nunca recibe CPU.

Error común: creer que 20 es la prioridad más alta.

5. Algoritmo 4: Round Robin (preventivo)

Para GPOS: cada tarea recibe un **cuanto** fijo, en cola circular. Escritorio: **4 ms**. Al vencer, el timer interrumpe y la tarea va de RUNNING a READY al final; cada cambio cuesta una **conmutación de contexto**.

- Da la ilusión de multitarea con un solo núcleo.
- No sirve en **tiempo real duro**: rompe el determinismo de la ley de control (sensado → filtrado → ley de control → actuación).

6. Round Robin con prioridad y preventivo

Árbol balanceado donde cada nodo guarda una **cola circular** de una misma prioridad.

1. Se busca el nivel más alto con tareas READY.
2. En ese nivel se alterna por Round Robin (4 ms).
3. Nadie recibe dos cuantos seguidos salvo que sea la única lista en su nivel.
4. Si una tarea de mayor prioridad pasa a READY, **prevé** (interrumpe) a la de menor.

Entre tareas de igual prioridad solo cambia por vencer el cuanto o por E/S.

7. Frecuencia del scheduler

Sistema	Frecuencia	Rebanada
RTOS	1 kHz	1 ms (deadlines rígidos)
Linux escritorio	250 Hz	4 ms
Linux servidor	100 Hz	10 ms

Linux 2.6.30x: se elige al compilar entre 100, 250, 300 o 1000 Hz.

- Servidor: una página PHP con consultas tarda **~6 ms**; con 10 ms se despacha de un tiro sin conmutaciones.
- Evaluar y conmutar cuesta, por ejemplo, 1 µs; a 1 kHz son **1 ms perdido por segundo**.

Error común: poner 1000 Hz en un servidor creyendo que sirve más rápido.

8. Scheduler de Linux y CFS

Linux 1.0–1.4: algoritmo simple no preventivo de Torvalds; desde el kernel **2.6.23**, **CFS** (*Completely Fair Scheduler*), con árbol **rojo-negro** y tiempo virtual (*vruntime*).

Política	Uso
SCHED_NORMAL / SCHED_OTHER	Escritorio interactivo (RR con prioridad y prevención)
SCHED_BATCH	Rebanadas largas, sin conmutación frecuente (compilaciones, respaldos, consultas masivas)
SCHED_IDLE	Prioridad ultra baja; solo cuando el CPU está ocioso (monitoreo ambiental)
SCHED_FIFO / SCHED_RR	Tiempo real, prioridades **1 a 99**, por encima de cualquier tarea de usuario

⚠️ NotebookLM dijo que RT 1 es más prioritaria que RT 99; en Linux real **99 es la más alta**.

Modos de prevención: *No Preemptive* (servidores), *Voluntary Preemption*, *Preemptive Kernel / Low-Latency Desktop*.

Error común: creer que SCHED_FIFO y SCHED_RR vienen activados por defecto; requieren recompilar o el parche PREEMPT_RT.

Contexto y pendiente

- Comentarios sobre el nuevo plan de estudios (desaparecen Compiladores, Sistemas Operativos, Lenguajes Formales y Física Cuántica).
- Próxima clase (sin confirmar): administración de memoria y controladores. Lab: traer las Raspberry Pi con Linux en consola para compilar el kernel. Sin tarea nueva.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Algoritmo	Cómo elige	Para recordar
FIFO simple	Por llegada, sin interrumpir	Una tarea desbloqueada va al final; 80–90 % de los embebidos
SJF	La más corta	Árbol: $\log_2 300 \approx 9$; WCET
Prioridad	La de mayor prioridad	Linux: −20 la más alta; inanición
Round Robin	Turnos por cuanto	4 ms (250 Hz); no sirve en tiempo real duro
RR con prioridad	Cola circular por nivel + prevención	Entre iguales, solo cambia el cuanto o la E/S

Frecuencias y CFS

Escritorio 250 Hz (4 ms), servidor 100 Hz (10 ms), RTOS 1 kHz. Más frecuencia = más sobrecarga. CFS: NORMAL, BATCH, IDLE y las de tiempo real FIFO/RR (1–99); en Linux real 99 es la más alta.